



JIMMA CITY ADMINISTRATION

Technology & Deployment Strategy

Technical stack, operational model, release, and continuity guide

Purpose: Practical reference for implementation, operations, training, and handover of the Jimma City Digital Street Address MIS.

Document: MIS-OPS-002

Prepared: 27 August 2026

System: Jimma City Digital Street Address MIS

1. Technology stack

Layer	Technology	Use in the system
Frontend	React, TypeScript, Vite	Single-page municipal operations interface and responsive field collection screens.
UI	Tailwind CSS, Radix UI primitives, Lucide icons	Consistent forms, navigation, dialogs, tables, badges, and responsive layouts.
Routing	Wouter	Application routes such as Dashboard, Properties, Map, Reports, Users, Setup, and Mobile Collection.
Data fetching	TanStack React Query	Query caching, mutations, loading states, and refetching for API data.
Forms and validation	React Hook Form, Zod	Validated property, user, setup, authentication, and review inputs.
Charts	Recharts	Dashboard trends, status distribution, kebele counts, and performance views.
GIS	Leaflet and OpenStreetMap	Map view of properties with latitude and longitude; no commercial map API key required.
Backend	Node.js, TypeScript, Express 5	REST API, business rules, authorization, uploads, reports, and integration endpoints.
Authentication	JWT, bcryptjs	Bearer-token sessions and secure password hashing. Tokens are stored by the web client.
Database	PostgreSQL, Drizzle ORM, drizzle-zod	Users, kebeles, streets, blocks, properties, approvals, property photos, and audit logs.
API contract	OpenAPI 3.1, Orval, generated React Query hooks and Zod schemas	Contract-first alignment between API routes, frontend hooks, and validation types.
Files	Multer-based image uploads	Stores and serves property evidence photos with category metadata.
Email	Nodemailer with SMTP	Password reset email delivery using configured SMTP credentials.
SMS	Africa's Talking SDK	Separate owner notifications for Kebele verification and City approval. Sandbox is for testing.
Logging	Pino and pino-http	Structured API request and operational logging.
Workspace	pnpm monorepo	Separates deployable artifacts from shared API client, API specification, and database libraries.

2. Main data entities

Users Username, password hash, name, email, phone, role, Kebele assignment, and active status.	Properties Owner, contacts, address components, property type, ownership, floors, GPS, status, and remarks.	Approvals Property, action, actor, remark, and timestamp for verification, approval, rejection, or resubmission.
--	---	--

Locations

Kebeles, streets, and blocks form the address hierarchy used by capture and code generation.

Photos

Property photo URL, filename, file type, category, uploader, and timestamp.

Audit logs

Actor, action, entity, before/after values, device/IP details, and timestamp.

3. External services and environment configuration

Configuration	Purpose	Required for
<code>DATABASE_URL</code>	PostgreSQL connection string	All API data operations
<code>JWT_SECRET</code>	Production signing secret for authentication tokens	Secure production authentication
<code>SMTP_FROM_EMAIL</code>	Sender address for password recovery emails	Password reset delivery
<code>SMTP_APP_PASSWORD</code>	SMTP provider app password; never paste into documentation or source code	Password reset delivery
<code>AT_USERNAME</code>	Africa's Talking account or sandbox username	SMS delivery
<code>AT_API_KEY</code>	Africa's Talking API key; sandbox and production keys differ	SMS delivery

SMS note: Africa's Talking sandbox confirms requests in the simulator; it does not represent production handset delivery. Use a registered sender ID, the correct environment key, and funded/approved production configuration for live notifications.

4. Technology hands-on orientation

1. Open the web application and sign in with an assigned role.
2. Use Dashboard to understand current data volume and status distribution.
3. Use Properties to search, filter, open a record, and follow its status history.
4. Use Map View to inspect GPS coverage and identify records without coordinates.
5. Use Reports to filter and export operational lists.
6. Use Setup only when authorized to maintain the kebele, street, and block hierarchy.
7. Use Audit Logs to review who changed or approved a record.

1. Deployment model

The system is deployed as two coordinated services: a React/Vite web service for the browser interface and an Express API service for business logic and data. A PostgreSQL database stores transactional data, while property photos are stored in the API upload area or an approved persistent storage target.



2. Release strategy

1 Prepare

Confirm the approved code version, database backup, environment values, SMS/email provider status, and an identified release owner.

2 Validate

Run workspace type checks and build checks. If the API contract changes, regenerate the client hooks and schemas before building the frontend.

3 Apply database changes

Apply schema changes to the target database using the approved migration/push process. Take a backup before destructive or structural changes.

4 Deploy services

Deploy/restart the API and web services using the managed workflows. Keep the API reachable at the `/api` path and keep the web application at the root path.

5 Smoke test

Open login, sign in, load Dashboard, open Properties, view Map, open Reports, and perform one controlled approval test using non-production data.

6 Monitor and hand over

Check the API health endpoint, application logs, database connectivity, upload access, password reset email, and SMS simulator/provider result.

3. Operational checklist

Area	Before release	After release
Database	Backup completed; connection and schema reviewed; retention policy confirmed.	Run a read-only count check and verify new records can be read and written.
Secrets	Use production-only values for database, JWT, SMTP, and SMS; keep them in managed secrets.	Confirm no secret appears in logs, screenshots, PDFs, or source control.
Users	Confirm administrator and approver accounts; disable unused demo users.	Test each operational role with least-privilege actions.

Notifications	Confirm owner phone format, sender configuration, and sandbox/production mode.	Trigger one controlled Kebele verification and City approval notification.
Media	Confirm upload location is persistent and backed up; check file size/type limits.	Upload and view one approved test photo.
Availability	Confirm web and API workflows are configured and restartable.	Check logs and <code>/api/healthz</code> ; document any warning.
Rollback	Identify the prior known-good release and database recovery point.	Use rollback only after assessing data changes and user impact.

4. Backup, monitoring, and continuity

- Back up PostgreSQL on a schedule appropriate for daily property collection; retain at least one off-service copy.
- Back up property evidence photos together with the database reference data, or use persistent object storage with its own retention policy.
- Monitor API errors, database connection failures, upload errors, email failures, and SMS failures separately.
- Maintain a service contact list for database, hosting, email, and SMS provider support.
- Practice restoring a backup before the system is considered production-ready.

Production safety: Change all demo passwords, set a strong `JWT_SECRET` , and use valid production provider credentials. Do not distribute demo credentials as real operational credentials.

5. Example run commands for maintainers

The managed project provides the following maintainer commands:

Command	Purpose
<code>pnpm --filter @workspace/api-server run dev</code>	Run the API service in development.
<code>pnpm run typecheck</code>	Check shared libraries and application packages.
<code>pnpm run build</code>	Run type checking and package builds.
<code>pnpm --filter @workspace/api-spec run codegen</code>	Regenerate API client and validation outputs after OpenAPI changes.
<code>pnpm --filter @workspace/db run push</code>	Apply approved development database schema changes.

Controlled reference document. Update this document when the approved workflow, roles, integrations, or deployment configuration changes.